

# Programación para Física y Astronomía

Departamento de Física.

---

Coordinadora: C Loyola

Profesores C Femenías / D Basantes

Primer Semestre 2025

Universidad Andrés Bello

Departamento de Física y Astronomía



Introducción y Contexto

Fundamentos de Pandas y POO

Ejercicios Prácticos

Conclusiones y Próximos Pasos

# Introducción y Contexto

---

- **Semana 7, Sesión 1 (Sesión 13):**
  - Exploramos gráficos avanzados en Matplotlib (subplots múltiples, histogramas, 3D).
  - Tuvimos un vistazo opcional a pandas para el manejo de datos tabulares.
- **Semana 7, Sesión 2 (Sesión 14):**
  - Realizamos un **problema evaluado** integrando NumPy y Matplotlib (matriz aleatoria, álgebra lineal, gráficas 3D e histograma).
  - Aclaremos dudas tras la entrega.
- **Objetivo de hoy:** Iniciar **programación orientada a objetos (POO)** en Python y repasar análisis básico de datos en NumPy/pandas.

- **Comprender** los fundamentos de la Programación Orientada a Objetos (POO) en Python.
- **Explorar** el uso de **pandas** para análisis y manipulación de datos.
- **Aplicar** operaciones básicas y avanzadas de pandas en contextos físicos y astronómicos.
- **Desarrollar** habilidades para combinar POO y pandas en problemas prácticos.

# Fundamentos de Pandas y POO

---

# Fundamentos de Pandas y POO $\ni$ ¿Por qué Pandas para Análisis de Datos?

- **DataFrames:** estructuras tabulares similares a Excel/hojas de cálculo
- **Lectura:** archivos CSV, Excel, JSON automáticamente
- **Operaciones:** filtrado, agrupación, estadísticas descriptivas
- **Limpieza:** manejo de datos faltantes, duplicados
- **Integración:** funciona perfectamente con NumPy y Matplotlib

## Para Física/Astronomía

Ideal para analizar datos experimentales, observaciones astronómicas, resultados de simulaciones.

# Fundamentos de Pandas y POO $\ni$ ¿Qué es pandas?

- **pandas** es una biblioteca de Python para análisis y manipulación eficiente de datos.
- Permite trabajar con datos tabulares (tipo Excel) de forma sencilla y poderosa.
- Ofrece estructuras clave: **Series** (vectores) y **DataFrame** (tablas).
- Esencial para ciencia de datos, física computacional y astronomía.

## Ventajas

- Lectura y escritura de archivos (CSV, Excel, etc.)
- Filtrado, selección y limpieza de datos
- Estadísticas y agrupaciones rápidas
- Integración con NumPy y Matplotlib



# Fundamentos de Pandas y POO $\ni$ Aplicación: Análisis de un archivo en línea con pandas

## Enunciado

- Puedes aplicar los conceptos de pandas usando archivos públicos en la web.
- Ejemplo: el archivo `titanic.csv` disponible en GitHub.
- Carga el archivo directamente desde la URL y explora sus primeras filas.

```
1 import pandas as pd
2
3 url = 'https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv'
4 df = pd.read_csv(url)
5 print(df.head())
```

**Discusión:** Así puedes practicar análisis de datos reales sin descargar archivos manualmente. Ideal para ejercicios en clase o tareas.

# Fundamentos de Pandas y POO $\ni$ Estructuras principales de pandas

- **Series**: arreglo unidimensional con etiquetas (índice).
- **DataFrame**: tabla bidimensional con filas y columnas etiquetadas.

```
1 import pandas as pd
2
3 # Serie
4 s = pd.Series([10, 20, 30], index=['a', 'b', 'c'])
5
6 # DataFrame
7 df = pd.DataFrame({
8     'masa': [1.5, 2.0, 0.8],
9     'velocidad': [10.5, 8.2, 15.3]
10 })
```

**Discusión:** Una **Series** es como un vector con índice; un **DataFrame** es como una tabla de Excel.

# Fundamentos de Pandas y POO $\ni$ Acceso y selección de datos en DataFrame

```
1 # Acceso por columna
2 print(df['masa'])
3
4 # Acceso por fila (iloc: por posición, loc: por etiqueta)
5 print(df.iloc[0])      # Primera fila
6 print(df.loc[1])       # Fila con índice 1
7
8 # Selección de subconjuntos
9 print(df[['masa', 'velocidad']])
10 print(df[0:2])         # Primeras dos filas
```

**Discusión:** pandas permite seleccionar datos de forma flexible y eficiente, facilitando análisis rápidos.

# Fundamentos de Pandas y POO $\ni$ Operaciones vectorizadas y estadísticas

```
1 # Operaciones sobre columnas
2 df['energia_cinetica'] = 0.5 * df['masa'] * df['velocidad']**2
3
4 # Estadísticas rápidas
5 print(df['energia_cinetica'].mean())
6 print(df.describe())
```

**Discusión:** Las operaciones se aplican a toda la columna sin necesidad de bucles explícitos.

# Fundamentos de Pandas y POO $\ni$ Limpieza y manejo de datos faltantes

```
1 import numpy as np
2 df.loc[1, 'velocidad'] = np.nan
3
4 # Detectar valores nulos
5 print(df.isnull())
6
7 # Eliminar filas con NaN
8 df_clean = df.dropna()
9
10 # Rellenar valores faltantes
11 df_filled = df.fillna(0)
```

**Discusión:** pandas facilita la detección y el tratamiento de datos incompletos, común en experimentos reales.

# Fundamentos de Pandas y POO $\ni$ Agrupación y resumen de datos

```
1 # Crear una columna categórica
2 df['categoria'] = pd.cut(df['masa'], bins=[0,1,2], labels=['Baja', 'Alta'])
3
4 # Agrupar y calcular promedio
5 print(df.groupby('categoria')['velocidad'].mean())
```

**Discusión:** Agrupar datos permite obtener estadísticas por categorías físicas (masas, temperaturas, etc.).

# Fundamentos de Pandas y POO $\ni$ Visualización rápida con pandas

```
1 import matplotlib.pyplot as plt
2
3 df['velocidad'].hist(bins=5)
4 plt.title('Histograma de velocidades')
5 plt.xlabel('Velocidad')
6 plt.ylabel('Frecuencia')
7 plt.show()
```

**Discusión:** pandas integra métodos de visualización para análisis exploratorio inmediato.

# Fundamentos de Pandas y POO $\ni$ Operaciones Básicas con DataFrames

```
1 import pandas as pd
2
3 # Crear DataFrame desde diccionario
4 data = {
5     'masa': [1.5, 2.0, 0.8, 1.2],
6     'velocidad': [10.5, 8.2, 15.3, 12.1],
7     'temperatura': [300, 350, 280, 320]
8 }
9 df = pd.DataFrame(data)
10
11 # Operaciones básicas
12 print(df.head())          # Primeras filas
13 print(df.info())          # Información del DataFrame
14 print(df.describe())       # Estadísticas descriptivas
15 print(df.shape)           # Dimensiones (filas, columnas)
```



```
1 # Seleccionar columnas
2 masas = df['masa']           # Una columna
3 subset = df[['masa', 'velocidad']] # Múltiples columnas
4
5 # Filtrado con condiciones
6 altas_temp = df[df['temperatura'] > 300]
7 rapidos = df[df['velocidad'] > 10]
8
9 # Filtrado múltiple
10 criteria = (df['masa'] > 1.0) & (df['velocidad'] > 10)
11 filtered_df = df[criteria]
12
13 # Agregar nuevas columnas calculadas
14 df['energia_cinetica'] = 0.5 * df['masa'] * df['velocidad']**2
15 df['momento'] = df['masa'] * df['velocidad']
```

# Fundamentos de Pandas y POO $\ni$ Agrupación y Estadísticas Avanzadas

```
1 # Crear categorías para agrupación
2 df['categoria_masa'] = pd.cut(df['masa'],
3                               bins=[0, 1, 2, np.inf],
4                               labels=['Baja', 'Media', 'Alta'])
5
6 # Agrupar y calcular estadísticas
7 grouped = df.groupby('categoria_masa').agg({
8     'velocidad': ['mean', 'std'],
9     'energia_cinetica': ['sum', 'max'],
10    'temperatura': 'mean'
11 })
12
13 print(grouped)
14
15 # Correlaciones entre variables
16 correlations = df[['masa', 'velocidad', 'temperatura']].corr()
17 print("Matriz de correlaciones:")
18 print(correlations)
```

# Fundamentos de Pandas y POO $\ni$ Manejo de Datos Faltantes y Limpieza

```
1 # Simular datos faltantes
2 df.loc[1, 'temperatura'] = np.nan
3 df.loc[3, 'velocidad'] = np.nan
4
5 # Detectar valores faltantes
6 print("Valores nulos por columna:")
7 print(df.isnull().sum())
8
9 # Opciones para manejar datos faltantes
10 df_dropped = df.dropna()           # Eliminar filas con NaN
11 df_filled = df.fillna(df.mean())   # Rellenar con promedio
12 df_interpolated = df.interpolate() # Interpolación
13
14 # Eliminar duplicados (si existen)
15 df_clean = df.drop_duplicates()
```

# Fundamentos de Pandas y POO $\ni$ Lectura de Archivos y Exportación

```
1 # Leer diferentes formatos
2 df_csv = pd.read_csv('datos_experimento.csv')
3 df_excel = pd.read_excel('mediciones.xlsx', sheet_name='Hoja1')
4
5 # Parámetros útiles para CSV
6 df_custom = pd.read_csv('datos.csv',
7                          sep=';',          # Separador
8                          decimal=',',      # Separador decimal
9                          encoding='utf-8',  # Codificación
10                         skiprows=2)         # Saltar filas
11
12 # Exportar resultados
13 df.to_csv('resultados_procesados.csv', index=False)
14 df.to_excel('analisis.xlsx', sheet_name='Datos')
15
16 # Exportar solo columnas específicas
17 df[['masa', 'energia_cinetica']].to_csv('energia_datos.csv')
```

# Fundamentos de Pandas y POO $\ni$ Integración con POO: Clase Particle

```
1 import math
2
3 class Particle:
4     def __init__(self, mass, x, y, vx, vy):
5         self.mass = mass
6         self.x = x
7         self.y = y
8         self.vx = vx
9         self.vy = vy
10
11     def kinetic_energy(self):
12         """Retorna la energía cinética de la partícula."""
13         v2 = self.vx**2 + self.vy**2
14         return 0.5 * self.mass * v2
15
16     def distance(self, other):
17         """Distancia a otra partícula."""
18         dx = self.x - other.x
19         dy = self.y - other.y
20         return math.sqrt(dx**2 + dy**2)
```

# Fundamentos de Pandas y POO $\ni$ Ejemplo de uso:

## Clase Particle



```
1 # Crear dos partículas con diferentes posiciones y velocidades
2 p1 = Particle(mass=2.0, x=0, y=0, vx=3, vy=4)
3 p2 = Particle(mass=1.5, x=1, y=2, vx=0, vy=5)
4
5 # Calcular energía cinética de cada partícula
6 print("Energía cinética p1:", p1.kinetic_energy())
7 print("Energía cinética p2:", p2.kinetic_energy())
8
9 # Calcular distancia entre ambas partículas
10 dist = p1.distance(p2)
11 print("Distancia entre p1 y p2:", dist)
```

**Discusión:** Este ejemplo muestra cómo crear objetos **Particle**, calcular su energía cinética y la distancia entre ellas, integrando conceptos de POO y física.

# Fundamentos de Pandas y POO $\ni$ Visualización Integrada con Pandas y Matplotlib

```
1 import matplotlib.pyplot as plt
2
3 # Pandas tiene métodos de plotting integrados
4 df.plot(x='masa', y='velocidad', kind='scatter',
5         title='Masa vs Velocidad')
6 plt.show()
7
8 # Histograma de una columna
9 df['energia_cinetica'].hist(bins=10, alpha=0.7)
10 plt.title('Distribución de Energía Cinética')
11 plt.xlabel('Energía Cinética')
12 plt.show()
13
14 # Gráfico de barras por categorías
15 df.groupby('categoria_masa')['velocidad'].mean().plot(kind='bar')
16 plt.title('Velocidad Promedio por Categoría de Masa')
17 plt.show()
```

# Ejercicios Prácticos

---



## Información del Dataset

- **Fuente:** <https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>
- **Contenido:** Datos de pasajeros del Titanic (891 registros)
- **Variables principales:**
  - **Age:** Edad del pasajero
  - **Fare:** Precio del boleto
  - **Pclass:** Clase de pasajero (1, 2, 3)
  - **Sex:** Género (male, female)
  - **Survived:** Supervivencia (0=No, 1=Sí)
  - **SibSp:** Número de hermanos/esposas a bordo
  - **Parch:** Número de padres/hijos a bordo
  - **Ticket:** Número de ticket
  - **Fare:** Tarifa pagada
  - **Cabin:** Cabina asignada
  - **Embarked:** Puerto de embarque (C = Cherburgo, Q = Queenstown, S = Southampton)

# Ejercicios Prácticos $\ni$ Ejercicio 1:

## Exploración básica de datos del Titanic



### Enunciado

- Cargar el archivo `titanic.csv` desde la URL pública.
- Mostrar las primeras 8 filas del DataFrame.
- Listar los nombres de todas las columnas.
- Obtener el número total de pasajeros y cuántos sobrevivieron.
- Calcular la edad promedio y la tarifa (**Fare**) promedio.

**Objetivo:** Practicar carga, inspección y estadísticas básicas con pandas usando un dataset real.

<https://raw.githubusercontent.com/datasciencedojo/datasets/master/titanic.csv>

## Ejercicios Prácticos $\ni$ Ejercicio 2: Filtrado y selección de datos



### Enunciado

- Filtrar los pasajeros de clase 1 (`Pclass == 1`).
- Seleccionar solo las columnas **Name**, **Sex**, **Age**, **Fare**.
- Mostrar los 5 pasajeros más jóvenes de clase 1.
- (Opcional) Calcular el porcentaje de mujeres en clase 1.

**Objetivo:** Aplicar filtrado y selección de columnas en pandas.

## Ejercicios Prácticos $\ni$ Ejercicio 3:

### Limpieza y manejo de datos faltantes



#### Enunciado

- Identificar cuántos valores faltantes hay en la columna **Age**.
- Crear una copia del DataFrame y rellenar los valores faltantes de **Age** con la edad promedio.
- Mostrar la edad mínima y máxima después de la limpieza.
- (Opcional) Eliminar filas donde falte el valor de **Embarked**.

**Objetivo:** Practicar detección y tratamiento de datos faltantes en pandas.

## Ejercicios Prácticos $\ni$ Ejercicio 4: Agrupación y análisis por categorías



### Enunciado

- Agrupar los datos por **Pclass** y calcular la tarifa promedio (**Fare**) y la edad promedio (**Age**) por clase.
- Calcular la tasa de supervivencia (**Survived**) por clase y por sexo.
- (Opcional) Mostrar los resultados en un gráfico de barras.

**Objetivo:** Aplicar agrupación y análisis estadístico por categorías usando pandas.

## Ejercicios Prácticos $\ni$ Ejercicio 5:

### Visualización y exportación de resultados



#### Enunciado

- Crear un histograma de la columna **Age** para visualizar la distribución de edades.
- Crear un gráfico de barras con el número de pasajeros por puerto de embarque (**Embarked**).
- Exportar un DataFrame con las columnas **Name**, **Age**, **Survived** a un archivo `titanic_resumen.csv`.
- (Opcional) Guardar una de las gráficas como imagen (`.png`).

**Objetivo:** Consolidar visualización y exportación de datos con pandas.

- **POO**: mantén las clases simples y bien comentadas.
- **pandas**: `df.iterrows()` o `df.itertuples()` puede ayudarte a iterar filas.
- **NumPy**: para la matriz de distancias, considera `np.zeros((n,n))` de base.
- **Visualización**: usa `plt.scatter(df['x'], df['y'])` si usas DataFrame.

## Conclusiones y Próximos Pasos

---



## Conclusiones y Próximos Pasos $\ni$ Discusión de Soluciones con pandas

- ¿Qué funciones de **pandas** te resultaron más útiles para explorar y limpiar el dataset del Titanic?
- ¿Cómo filtraste y seleccionaste subconjuntos de datos relevantes?
- ¿Qué dificultades encontraste al manejar datos faltantes o agrupar información?
- ¿Qué visualizaciones generaste y cómo interpretaste los resultados?
- ¿Exportaste algún resultado? ¿Cómo lo hiciste con pandas?

### Objetivo

Compartir estrategias y buenas prácticas usando pandas para análisis de datos reales.

- Aplicamos **pandas** para análisis y manipulación de datos reales (**titanic.csv**).
  - Carga, inspección, filtrado y limpieza de datos.
  - Agrupación, estadísticas y visualización rápida.
- Reforzamos la importancia de pandas en el flujo de trabajo científico y en la exploración de grandes datasets.
- Practicamos la exportación de resultados y la generación de gráficas directamente desde pandas.
- Próximamente: integración de pandas con otras herramientas y análisis más avanzados.

# Gracias y hasta la próxima sesión

- Practiquen la creación de **clases** y la **lectura** de datos con **pandas**.
- ¡Nos vemos en la **Semana 8, Sesión 2** para profundizar POO!